Week 3 quiz solutions

Contents

Questions and solutions 1
Quiz statistics 14

Questions and solutions

This section lists all questions in the pool from which quiz questions were drawn. The pool contains 27 questions, from which 20 were selected at random for each student.

Question: Which of the following are not amongst the traditional information security goals?

1. Confidentiality
2. Privacy
3. Resilience
4. Integrity
5. Availability

Answer:

The correct answer is options 2 and 3, "privacy" and "resilience". The traditional information security goals are confidentiality, integrity and availability ("C I A"). See chapter 1 of the textbook and slide 32 of the week 1 lecture.

Privacy and resilience are not amongst these goals.

Question: *Authenticity* is

1. being confident in or able to verify the genuineness of a message or information
2. being true to one's own self
3. the ability to trace actions back uniquely to the entity that took those actions
4. the creation of evidence that an action has occurred, so that a user cannot later falsely deny taking that action

Answer:

The correct answer is option 1, "being confident in or able to verify the genuineness of a message or information". Refer to slide 38 of the week 1 lecture. The textbook doesn't list authenticity as a goal, but does describe "authentication" as being one of the primary means to achieving the "C I A" goals, and defines "authentication" as "High-assurance determination of the identity of a principal".

Of the incorrect options, option (2) has nothing to do with information security; option (3) is actually a description of "accountability"; and option (4) is actually a description of "non-repudiation".

Question: *Attacks* are a superset of *incidents* , true or false?

Answer: The correct answer is "False". *Attacks* require intent, whereas security *incidents* could be human-caused but accidental (e.g. accidentally erasing records) or the result of natural threats (e.g. fire or flood). Attacks therefore can't possibly be a superset of "incidents"; they're a subset.

Question: The size of a `long` on the standard CITS3007 development environment is how many bytes?

1. 4 bytes
2. 2 bytes
3. 8 bytes
4. 1 byte
5. 16 bytes

Answer: The correct answer is "8 bytes". This can be verified by executing the following program on the CITS3007 development environment:

```c
#include <stdio.h>
#include <stdlib.h>

// The size of a `long` on the standard CITS3007 development
// environment is how many bytes?
int main() {
  printf("sizeof long: %zd\n", sizeof(long));
}
```

The output when this program is run is: `sizeof long: 8`.

Another way to check the answer is to start `gdb` in the development environment, load any compiled binary, and execute the command `print sizeof(long)`; gdb will print

```
$1 = 8
```

Note that gdb determines CPU architecture based on the program you're debugging, and if you *don't* load a program in `gdb`, then it defaults to assuming a 32-bit system and will report incorrect sizes for some data types (e.g. it reports 4 bytes for a `long`). To explicitly set the correct CPU architecture without loading a 64-bit program, you can issue the following command in `gdb`:

```
set architecture i386:x86-64
```

Question: Which of the following are *not* integer data types?

1. `double`
2. `float`
3. `bool`
4. `int`
5. `short int`
6. `char`

Answer: The correct answers are "`double`" and "`float`": these are not integer data types, but are instead "real floating types". All the rest are integer data types (in fact, they are amongst the *standard* integer types): see slide 8 of the week 2 lecture slides. As that slide states, nearly *every* type we use in this unit will be either an integer data type, or derived from an integer data type (or types).

(The standard integer types are defined in the ISO/IEC C11 standard (PDF) in section 6.2.5, "Types", para 7.)

Question: Which of the following are signed integer data types?

1. `signed char`
2. `int`
3. `bool`
4. `unsigned int`
5. `long`

Answer: The correct answer is that all of these are signed integer data types except for `bool` and `unsigned int`, which are unsigned.

Refer to slide 8 of the week 2 lecture, which lists the standard unsigned and signed integer data types in C.

(The signed and unsigned integer types are defined in the ISO/IEC C11 standard (PDF) in section 6.2.5, "Types".)

Question: Which CITS3007 facilitator has a beard?

1. Santiago
2. Arran
3. Dan
4. Chris
5. Rachel

Answer: The correct answer is "Santiago". Answering this question requires you to make use of one of the key information resources provided to you, the unit website. The website is extremely small (4 pages), and the information required is about halfway down the "Welcome" page, under the heading "Who'll be helping in labs".

Alternatively, Googling for "`"cits3007" "facilitators"`" returns only a single relevant hit (the welcome page for the unit site), and searching within that page for "facilitators" will show photographs of both.

Question: Suppose you are on a platform where an `int` is 2 bytes in size. How many bytes would the call `malloc(sizeof(int)*10)` allocate?

Answer: The correct answer is 20. You are told that `sizeof(int)` on the specified platform is 2; therefore `sizeof(int)*10` is 20, and `malloc` will allocate that many bytes.

The documentation for the `malloc` function can be viewed by running `man malloc` in the development environment, or going to https://en.cppreference.com/w/c/memory/malloc.

Question: Suppose you are on a platform where an `int` is 4 bytes in size, and you have two variables of type pointer to int, `p1` and `p2`. The value of `p1` is `0x104` (decimal 260), and the value of `p2` is `0x110` (decimal 272).

What will be the result of the expression `p2 - p1`?

Answer: The correct answer is 3.

Pointer arithmetic returns answers based on the *size of the data-type being pointed to* (refer to slide 19 of the week 2 lecture slides). Therefore `p2 - p1` will tell you how many `int` elements will fit between `p2` and `p1`. We know that an `int` is 4 bytes in size on the specified platform, therefore the result will be $(272 - 260)/4$, which equals 3.

One can check the answer to this by writing a short C program. As it happens, the size of an `int` on the standard CITS3007 development environment is, in fact, 4 bytes, so the following program will print the answer:

```
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
```

```c
4  #include <assert.h>
5  #include <stdint.h>
6
7  // Suppose you are on a platform where an int is 4 bytes in size, and
8  // you have two variables of type pointer to int, p1 and p2. The value
9  // of p1 is 0x104 (decimal 260), and the value of p2 is 0x110 (decimal
10 // 272).
11 //
12 // What will be the result of the expression p2 - p1?
13
14 // confirm that an int is 4 bytes
15 // on the standard cits3007 dev environment
16 static_assert(sizeof(int) == 4, "size of int must be 4 bytes");
17
18 int main() {
19   printf("sizeof int: %zd\n", sizeof(int));
20   int * p1 = (int*) 0x104; // decimal 260
21   int * p2 = (int*) 0x110; // decimal 272
22   ptrdiff_t diff = p2 - p1;
23   printf("value of diff: %td\n", diff);
24 }
```

The output when this program is run is:

```
sizeof int: 4
value of diff: 3
```

The program will also compile and run correctly if lines 22–23 are replaced with the single statement:

```
   printf("value of diff: %ld\n", p2 - p1);
```

(`ptrdiff_t` is the type of the result when one pointer value is subtracted from another – see https://en.cppreference.com/w/c/types/ptrdiff_t – but on the CITS3007 development environment, it's possible to get by with treating it as being the same as a `long`, since it's an 8-byte signed integer, just like a `long` is.)

It's also possible to check this by using `gdb` and running the command

```
print (int*) 0x110 - (int*) 0x104
```

`gdb` will print the result `$1 = 3`.

Question: Suppose a variable is declared as follows:

```
long int * * myvar;
```

What is the data type of `myvar`?

1. pointer to pointer to `long`
2. pointer to `long`
3. pointer
4. pointer to pointer to pointer to `long`
5. pointer to `long int`

Answer: The correct answer is "pointer to pointer to `long`". We form a pointer type in C by putting an asterisk ("`*`") after some other type; there are two asterisks here, so the return type must therefore be "pointer to pointer to `long int`". "`long int`" in C is just another name for "`long`".

Option 3 cannot be correct, because "pointer" isn't a data-type in C. Options 2 and 5 can't be correct, because they're only "pointers", not "pointers to pointers". Option 4 can't be correct, because it has too many levels of indirection.

(A useful website for translating between English and C declarations is https://cdecl.org – try pasting "`long int * * myvar`" into the input box on that site, and see what output you get.)

Question: The C11 standard guarantees that a `char` is 8 bits in size, true or false?

Answer: The correct answer is "false". See slide 9 of the week 2 lecture slides: the C11 standard requires that a `char` has *at least* 8 bits, not that it *is* 8 bits in size. You should be able to understand phrases like "at least" and "at most", and know that both of these mean something different to "is equal to".

Question: Is the following a function declaration, or a function definition?

```
long int** f(char * x, int y);
```

1. declaration
2. definition
3. neither

Answer: The correct answer is "declaration". See slide 13 of the week 2 lecture slides: no function body is provided here, so this cannot be a function *definition*.

Question: Suppose you are on a platform where a `long` is 4 bytes in size, and you have the following declarations at file level:

```
long * p1 = (long *) 0x1100;
long * p2 = p1 + 4;
```

What will be the result of the expression `(p2 - p1) * 2 + 1`?

Answer: The correct answer is 9.

Pointer arithmetic returns answers based on the *size of the data-type being pointed to* (refer to slide 19 of the week 2 lecture slides). Therefore `p2` will point to an address 4 "`long`-widths" past `p1`, and `p2 - p1` will evaluate to 4.

Therefore `(p2 - p1) * 2 + 1` will evaluate to $4 * 2 + 1$, or in other words 9.

Question: Which of the following are scopes which a function could have?

1. global scope
2. block scope
3. class scope

Answer: The correct answer is "global scope". (Refer to slide 14 of the week 2 lecture slides.) A function can't have block scope, because C doesn't have nested functions: all functions in C have file-level or "global" scope. (Though note that the `static` keyword can be used to make a function only visible in the file it is declared in.)

C does not have classes, so option 3 can't possibly be correct.

(In marking this question: the question could be seen as ambiguous, as a function can certainly *contain* variables which have block scope. Therefore answers which gave option 2 in addition to option 1 were awarded full marks for this question.)

Question: Suppose you are on a platform where an `int` is 2 bytes in size, and a file contains the following code.

```
#define W 10
#define W 20
#define H 5

int myarray[H][W];
```

How many bytes would the array `myarray` occupy?

Answer: The correct answer is 200. The declaration `int myarray[H][W]` will be equivalent to `int myarray[5][20]`. Therefore `myarray` is a contiguous block of memory holding 5×20 `int`s; since an `int` is 2 bytes in size, it follows that `myarray` occupies $5 \times 20 \times 2$ or 200 bytes.

It's possible to verify this using a short C program. The size of an `int` on the standard CITS3007 development environment is not 2 bytes but 4 bytes, so instead of an `int`, we need to use a type that *is* 2 bytes in size. An example of such a type is `int16_t`, which is defined in the `stdint.h` header (see `man stdint.h` or https://en.cppreference.com/w/c/types/integer).

```c
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
#include <assert.h>
#include <stdint.h>

// "mock" a 2-byte int by using an int16_t.
typedef int16_t myint;

// confirm that myint is 2 bytes in size
static_assert(sizeof(myint) == 2, "size of myint must be 2 bytes");

#define W 10
#define W 20
#define H 5

myint myarray[H][W];

int main() {
  printf("size of myarray (in bytes): %zd\n", sizeof(myarray) );
}
```

The output when this program is run is:

```
size of myarray (in bytes): 200
```

Question: What is the type of the following expression:

```
  "dog rat cat"
```

1. `char[12]`

2. `char[11]`
3. `char *`
4. `char[]`
5. `const char *`

Answer: The correct answer is `char[12]`. In C, there is no "string" type; rather, strings are arrays of `char`s. (See slide 17 of the week 2 lecture slides.) The string literal `"dog rat cat"` is an array with 12 elements (11 elements for each character in the literal, plus one for the `NUL` character).

Option 2 is incorrect, because it doesn't account for the terminating `NUL` character. Option 3 is incorrect, because it's simply the incorrect type. Option 4 is incorrect, because `char[]` is not a complete type: in C, arrays have a size which is fixed at compile-time. Option 5 is incorrect for the same reasons as option 3.

One can check the type of the string literal by executing the command `ptype "dog rat cat"` in `gdb`. `gdb` will output `type = char [12]`.

Question: Assume the following three variables are defined:

```
int i = 3;
int * p1 = &i;
int * p2 = (int*) 0x1100;
```

Which of the following are *not* valid pointer arithmetic expressions?

1. `p1 + p2`
2. `p2 - p1`
3. `p1 - p2`
4. `p1 + i`
5. `p1 - i`

Answer: The correct answer is option 1, because it tries to add a pointer to a pointer, and that's not a valid use of pointers. The other options are all valid uses of `p1` and `p2`. Two pointers of the same type can be subtracted (options 2 and 3), and one can also add a pointer and an integer (option 4), or subtract an integer from a pointer (option 5).

One can test the different possibilities by writing a short C program similar to the following:

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
  int i = 3;
  int *p1 = &i;
  int *p2 = (int*) 0x1100;
```

```

  printf("result of pointer arithmetic: %td\n", p1 + p2 );
}
```

Trying to comnpile the above program with `gcc` results in the following error message:

```
invalid_ptr_arith.c: In function 'main':
invalid_ptr_arith.c:9:52: error: invalid operands to binary +
(have 'int *' and 'int *')
    9 |   printf("result of pointer arithmetic: %td\n", p1 + p2 );
      |                                                       ^
```

All the other pointer arithmetic expressions, on the other hand, will compile with no errors.

Question: Assume the following code is contained in a file:

```
  struct person { int age; double height};
  struct person p { 32, 42 };
  struct person * pp = &p;
```

Which of the following expressions would evaluate to 32?

1. `p.height + 8`
2. `pp.age`
3. `pp->age`
4. `pp.height`
5. `pp->height`

Answer: The correct answer is option 3, `pp->age`.

Option 1 is incorrect; the value of `p.height + 8` is $42 + 8$ or 50. Options 3 and 4 are incorrect; since `pp` is of type "pointer to `struct person`", one can't use the member access operator ("`.`") with it. Option 5 is incorrect, because the value of `pp->height` is 42.

Once can test the various options by writing them in a small C program. For instance, one can test option 3 in a program like the following:

```c
#include <stdio.h>
#include <stdlib.h>

struct person {
  int age;
```

```
  double height;
};

int main() {
  struct person p = { 32, 42 };
  struct person * pp = &p;

  printf("age is: %d\n", pp->age );
}
```

The program will output 32, indicating that the 3rd option is correct.

Question: A "dangling pointer" is ... what?

1. The same as a hanging chad
2. A pointer to a variable which has gone out of scope
3. A pointer to a variable whose lifetime has ended
4. The result of a call to `malloc`
5. The result of memory allocated with `malloc` , but never `free` d

Answer: The correct answer is option 3, "A pointer to a variable whose lifetime has ended". (See slides 23 and 24 of the week 2 slides.) Options 1 and 4 are incorrect, because they have no relation to hanging pointers at all. Option 5 doesn't describe a hanging pointer, but rather a memory leak. Option 2 is closer to being correct, but "dangling pointers" are to do with the fact that the referent of the pointer is not valid because its *lifetime* has been exceeded, rather than the fact the variable has gone out of scope.

For most local variables, the two will coincide, but not always. C permits local variables to be declared `static`, which gives them the same lifetime as global variables: they persist for the lifetime of the process (and maintain their value between function calls).

Thus using the following function does not result in undefined behaviour:

```
1  int *myfunc() {
2    static int x;
3    return &x;
4  }
```

The variable `x` goes out of *scope* after line 4, but because its *lifetime* hasn't ended, the return value of `myfunc` is not a dangling pointer, and static analyzers like `clang-tidy` and `splint` will therefore not raise a warning.

Question: What is the return type of the function with the following declaration?

```
int * x(char * y);
```

Answer: The correct answer is "pointer to `int`". The answers "`int` pointer" and "`int *`" are also acceptable. "Pointer to integer" is accepted, but the previous answers are better because they are more specific: C has many integer data types. "Pointer" on its own is not sufficient: pointers in C are not in themselves a type.

Question: Which of the following functions frees dynamically allocated memory?

1. `malloc`
2. `calloc`
3. `realloc`
4. `free`

Answer: The correct answer is either 4, `free`, or both 3 and 4, `realloc` and `free`: either answer is acceptable.

`realloc` will not *always* free dynamically allocated memory, but will do so if asked to change the size of a previously allocated memory block to 0. Thus either answer is acceptable (since the question could be interpreted as asking "Which of the following functions (always) frees dynamically allocated memory?", or as asking "Which of the following functions (in at least some situations) frees dynamically allocated memory?".)

The question can be answered by looking up `man malloc` (or `man calloc`, or `man realloc`, or `man free` – on Ubuntu systems like the CITS3007 development environment, all these functions are documented on the same `man` page), and reading the function descriptions.

Options 1 and 2 are incorrect, because they don't free memory under any circumstances.

Question: Suppose you run a website, and an attacker is able to bring the website down by exploiting a buffer overflow vulnerability. In this case, which security goal would be compromised?

1. availability
2. confidentiality
3. integrity
4. authenticity
5. security
6. reliability

Answer: The correct answer is option 1, "availability". If the attacker takes the website offline, then it's no longer available to users.

Question: Which of the following is a methodology for categorizing threats?

1. STRIDE
2. WALK
3. RUN
4. PACE
5. STALK
6. STOMP

Answer: The correct answer is option 1, "STRIDE". The STRIDE methodology is discussed in chapter 1 of the textbook (part of the reading for week 1).

(It's also possible to answer this question by simply Googling "STRIDE security methodology", "WALK security methodology", etc. The results should reveal that only STRIDE is the name of a security methodology.)

Question: The text segment of a program in memory, which contains the processor instructions for the program, can normally be read from but not written to -- true or false?

Answer: The correct answer is "true". (Refer to the lecture discussion of slide 31 from the week 2 lecture slides.)

On most modern platforms, it's possible to mark particular memory segments as being read-only, and the text segment is one of the segments normally marked this way.

Question: Fill in the blanks: each time a [a] begins executing, a [b] is pushed onto the call stack.

1. function
2. stack frame
3. call stack
4. method
5. return address

Answer: The correct answer: "Each time a **function** begins executing, a **stack frame** is pushed onto the call stack." Refer to slide 29 of the week 2 lecture slides.

Question: Suppose you have an array which is declared as follows:

```
char myarray[3][1][2];
```

How many dimensions does the array `myarray` have?

Answer: The correct answer is 3. In C, as in many other languages, each set of brackets in the array declaration indicates a dimension.

Question: In C, the `char` type is a signed integer type -- true or false?

Answer: The correct answer is "false". Refer to slide 9 of the week 2 lecture slides. In C, the `char` type is *not* a signed integer type, nor is it an unsigned integer type. Rather, it falls into its own category, and whether it is equivalent to a `signed char` or an `unsigned char` is platform-specific.

Quiz statistics

The average mark (as well as the median mark) was 15 out of 20 (75%), with a standard deviation of 2.45 marks. A score of 17 or higher puts you in the top quartile of students, which is excellent; a score of 13 or lower puts you in the bottom quartile, which suggests you may need to take a different approach to learning CITS3007 material.

The questions most frequently answered incorrect were

- "A 'dangling pointer' is ... what?"

  Many people answered that a dangling pointer occurs when a variable goes out of scope.

- "... Which of the following are not valid pointer arithmetic expressions?"

  Many people selected other options, in addition to option 1.

- "What is the type of the following expression: `"dog rat cat"`?"

  Many people selected incorrect options; the most commonly selected incorrect option was `char *`.

For each question, Blackboard calculates how good a predictor the question is of overall score. (It calculates the Pearson correlation coefficient between the scores for that question and overall quiz scores.) The question which was the best predictor of overall score (with a correlation coefficient of 0.46) was:

> Which of the following are *not* integer data types?
>
> 1. `double`
> 2. `float`
> 3. `bool`
> 4. `int`
> 5. `short int`
> 6. `char`

Of the top 25% of students, none selected `bool`, `int`, `short int`, or `char`.

The next best predictor of overall score (with a correlation coefficient of 0.43) was "The size of a `long` on the standard CITS3007 development environment is how many bytes?".

My hypothesis would be that students who did well in the quiz probably

(a) read the slides, since the slides clearly state what is and isn't an integer data type
(b) wrote C programs to check their answers (since that seems to be the easiest way to work out the size of a `long`).

Whether a student has successfully passed CITS2002 doesn't seem to have any correlation with how well they did in the quiz (I calculated the correlation at about 0.1, which suggests no or at most a very weak relationship between the two[1]).

[1] Or that I made a mistake in my data analysis – which is certainly possible, as I haven't checked my code very thoroughly.